# Aircraft Hydraulic Leak Detection System

Machine-learning leak detection, quantification and localisation in a miniature hydraulic loop using pressure, flow and acoustic-emission sensor fusion

**Project Report — Group 32**

Design Practicum (IC-202P22)

Indian Institute of Technology Mandi, Himachal Pradesh


| Member | Roll no. | Branch |
|---|---|---|
| Aman Kumar | B25119 | CSE |
| Ashish Kumar | B25202 | DSAI |
| Badavath Thirumalesh | B25259 | EE |
| Bhoomika | B25347 | GE |
| Ananya Sharma | B25374 | MSE |
| Nishant Thamman | B25476 | ME |

## Abstract

Undetected leakage in hydraulic pipelines wastes fluid, lowers system pressure and, in safety-critical applications such as aircraft hydraulics, can disable actuation systems. This project designs a low-cost, laboratory-scale system that detects, quantifies and localises leaks in a miniature closed-loop hydraulic rig carrying ISO VG32 oil. Two pressure sensors and two flow sensors at the loop inlet and outlet capture the mass-balance and pressure signature of a leak, while four piezoelectric (PZT) discs mounted on steel saddles capture the high-frequency noise of oil jetting through the leak. An ESP32 microcontroller acquires pressure (through a 16-bit ADS1115 converter) and flow, an STM32F411 samples the four amplified acoustic channels at 200 kHz each and reduces them to nine features per channel, and a laptop logs labelled experiments, extracts about sixty features per one-second window and runs the trained model on a live dashboard. Four classifiers — Logistic Regression, SVM, Random Forest and XGBoost — are compared on a four-class task (no leak or one of three leak locations) with grouped cross-validation split by experimental run, and a staged pressure / flow / acoustic experiment quantifies what each sensor type contributes. A regression model estimates leak rate, and a persistence rule on the classifier output rejects transient false alarms.


**Keywords:** hydraulic leak detection; sensor fusion; acoustic emission; machine learning; Random Forest; ESP32; STM32

## 1. Introduction and Problem Statement

Aircraft hydraulic systems move flight-control surfaces, landing gear and brakes with high-pressure fluid. A leak caused by seal wear, a cracked fitting or pipe fatigue produces a measurable pressure drop, a change in flow and, in most cases, high-frequency acoustic emission from turbulent fluid escaping through the leak path.

Aircraft typically monitor system-level quantities such as pressure and reservoir level. These reveal that a fault exists but not where it is, so technicians must locate leaks manually in dense, visually obstructed hydraulic bays. A single sensing modality is also unreliable: pressure and flow respond slowly to small leaks and also change with normal operation, while acoustic sensing alone is sensitive to background mechanical noise.

This project therefore combines pressure, flow and acoustic sensing with supervised machine learning to detect, quantify and localise leaks in real time on a laboratory-scale rig. It is a maintenance-assistance demonstrator, not an aircraft-certified system.

## 2. Objectives

| # | Objective | How the project meets it |
|---|---|---|
| 1 | Zone-based pressure and flow differential sensing | 2 pressure + 2 flow sensors at loop inlet and outlet; 4 PZT discs, one per steel segment |
| 2 | Detection and quantification of leakage | Mass balance Q1 − Q2 plus an ML leak-rate regressor trained on measured leak volumes |
| 3 | Localisation to a zone; discrimination from normal changes | 4-class classifier trained with normal-but-disturbed runs; 2-second alarm persistence |
| 4 | Definition of sensing parameters | Sensor selection, sampling rates, accuracy, response time and 3σ minimum detectable leak (Sections 5 and 7) |
| 5 | Real-time alerting and logging; bench tests with VG32 oil | Streamlit dashboard with map, alarm and event log; ≈70 labelled runs in VG32 oil |

## 3. System Architecture

The system has three layers. Sensors on the pipe convert leak effects into electrical signals; two microcontrollers digitise them; and Python on a laptop records data, trains models and runs the live dashboard (Figure 1).

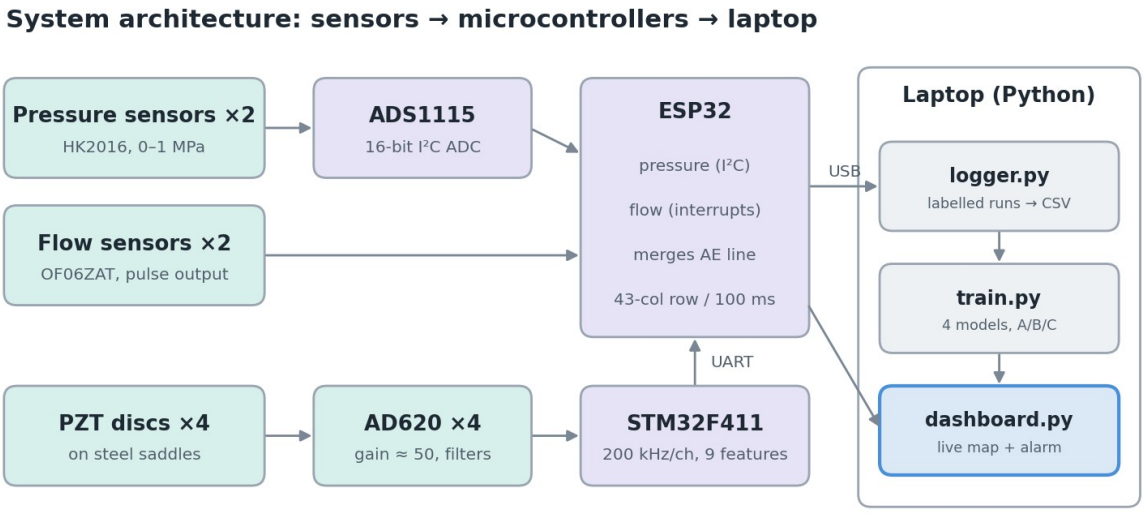



*Figure 1. System architecture from sensors to dashboard.*

Pressure and flow change slowly, so about 150 samples per second per channel is sufficient. The PZT signal lies around 40 kHz and must be sampled at about 200 kHz per channel. Sampling both on one microcontroller would cause timing conflicts, so the work is split: the **ESP32** handles pressure and flow deterministically and is the single link to the laptop, while the **STM32F411** samples the four acoustic channels with ADC and DMA, reduces each 100 ms of raw data to nine numbers per channel, and sends them to the ESP32 over UART. The ESP32 merges everything into one 43-value row every 100 ms and sends it to the laptop over USB.

Compared with the mid-semester design, the laptop replaces the Raspberry Pi as the processing platform, the ADS1115 replaces the ESP32's internal converter for pressure, and the STM32F411 with AD620 amplifiers provides the high-speed acoustic acquisition chain.

# 4. Experimental Test Rig

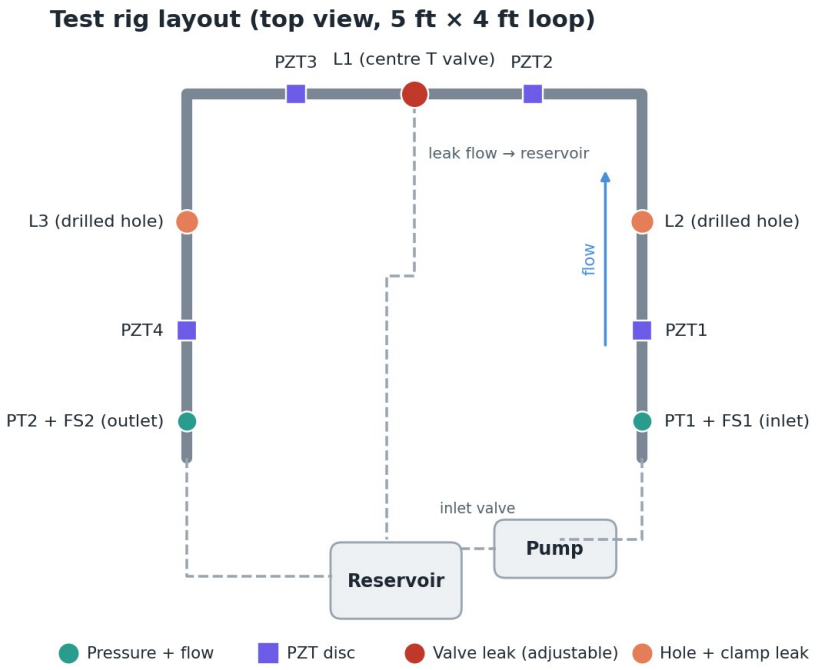



*Figure 2. Test rig layout with sensor and leak positions (positions to be finalised by measurement).*

| Parameter | Value |
|---|---|
| Pipe | 12 mm OD mild steel, 1 mm wall; rectangular loop ≈ 5 ft × 4 ft; hose couplings between segments |
| Fluid | ISO VG32 hydraulic oil (commissioning with water) |
| Pump | 12 V, 60 W self-priming diaphragm pump, up to 5 L/min (water); speed set by a 12–40 V, 10 A PWM controller |
| Flow path | Reservoir → inlet valve → pump → PT1/FS1 → right, top, left edges → FS2/PT2 → reservoir |
| Leak L1 | Ball valve on the centre T joint, hose back to reservoir; adjustable (¼, ½, full) |
| Leaks L2, L3 | 2–3 mm drilled holes on the right and left edges, sealed by rubber strip and hose clamp |
| Acoustic sensors | One PZT disc per steel segment on a grooved steel saddle |

The inlet valve stays fully open during runs, since throttling it starves the pump and creates bubbles that the acoustic sensors would mistake for leaks; flow is changed with the pump's PWM controller instead.

# 5. Hardware and Circuits

## 5.1 Sensors

| Sensor | Model | Range / output | Accuracy | Read by |
|---|---|---|---|---|
| Pressure ×2 | HK2016 | 0–1 MPa → 0.5–4.5 V (5 V supply); response ≤ 2 ms | ±1 %FS | ADS1115, 16-bit |

| Sensor | Model | Range / output | Accuracy | Read by |
|---|---|---|---|---|
| Flow ×2 | OF06ZAT | 30–3000 L/h; NPN pulses, F = 1.2·Q (water) | ±1 % (water) | ESP32 interrupts |
| Acoustic ×4 | 50 × 2.6 mm PZT, 40 kHz | mV-level AC, resonant | Uncalibrated | AD620 → STM32 |

## 5.2 Pressure channel

Each pressure output passes through a 10 kΩ / 20 kΩ divider that scales 0.5–4.5 V to 0.33–3.0 V, then a 1 kΩ resistor and 100 nF capacitor (≈200 Hz filter) into ADS1115 inputs A0 and A1. A third divider on A2 measures the 5 V rail, because the sensor output is ratiometric:

$$V_{out} = V_{cc} \cdot (0.8 \cdot P + 0.1), \quad P \text{ in } MPa$$

The ADS1115 runs from 3.3 V, with SDA on GPIO21, SCL on GPIO22 and ADDR to GND (address 0x48).

## 5.3 Flow channel

Each flow sensor output (NPN open collector) has a 10 kΩ pull-up to 3.3 V followed by a 1 kΩ / 100 nF filter into GPIO25 or GPIO26. The firmware measures the period between pulses, which stays accurate at the low pulse rates of this sensor, and ignores pulses closer than 2 ms. The K-factor (Hz per L/min) must be recalibrated with oil using a measuring jug.

## 5.4 Acoustic channel

Each PZT channel protects, amplifies, re-centres and filters the disc signal before the STM32 converter:

| Ref | Part | Purpose |
|---|---|---|
| R1, R2 | 1 kΩ in each PZT lead | Limit current from impact spikes |
| R3, R4 | 1 MΩ from each input to GND | DC bias path for the AD620 inputs |
| D1–D4 | 1N4148, back-to-back pairs | Clamp inputs to ±0.7 V |
| U1 | AD620 module, gain ≈ 50 | Differential amplification; rejects noise common to both leads |
| C1 | 2.2 nF C0G, series | Block DC offset; with R5/R6 a ≈14 kHz high-pass |
| R5, R6 | 10 kΩ / 10 kΩ from 3.3 V | Centre the signal at 1.65 V |
| R7, C2 | 1 kΩ series, 1 nF to GND | Anti-alias low-pass ≈100 kHz; charge reservoir for the ADC |
| D5 | BAT54S | Clamp the ADC input to 0–3.3 V |

The gain is kept near 50 because the AD620 bandwidth falls with gain (about 120 kHz at G = 100 and 12 kHz at G = 1000, per the AD620 datasheet), using

$$G = 1 + 49.4\ k\Omega \,/\, R_G$$

A typical leak signal of ±5 mV becomes about ±250 mV after amplification and is read as 1.40–1.90 V. All values are starting points to be confirmed with a single-channel oscilloscope test before building four channels.

## 5.5 PZT mounting

A flat 50 mm disc touches a 12 mm pipe only along a line, so each disc is bonded with thin epoxy to a 70 × 50 × 10 mm mild-steel saddle with a 12 mm, 5 mm-deep groove, greased and clamped to the pipe

with two hose clamps over the saddle ends. All four mounts must be identical because localisation compares loudness between discs. The discs are resonant 40 kHz transducers, so the system relies on relative changes rather than absolute frequency measurements.

### 5.6 Pump drive and power

The pump is driven by a PWM controller (Power+ from 12 V through a 10 A fuse; Motor+ and Motor− to the pump). Motor− switches and must never be connected to electronics ground. A 12 V, ≥ 8 A supply is used, with a single star ground at its negative terminal shared by the ESP32, STM32, sensors and buck converter. PWM switching (typically around 13 kHz) and its harmonics lie in the acoustic band, so leak experiments are run at 100 % duty where possible, with twisted pump wires and a ferrite.

## 6. Software and Data

| Component | Role |
|---|---|
| ESP32 firmware (Arduino) | Pressure via ADS1115, flow via interrupts, merges STM32 features, 43-column row every 100 ms |
| STM32 firmware (CubeIDE + CMSIS-DSP) | 200 kHz/channel ADC + DMA, 9 features per channel per 100 ms, UART at 230400 baud (specified) |
| logger.py | Records one labelled run to CSV + JSON |
| features.py | 1-second windows (50 % overlap) → ≈60 features; shared by training and live inference |
| train.py | Compares four models, runs the A/B/C fusion experiment, saves the best model and charts |
| dashboard.py (Streamlit) | Live map, alarm, plots and event log; replay mode for recordings |
| simulate.py, ae_reference.py | Synthetic data for testing; reference definition of the AE features |

Each row contains time, inlet and outlet pressure (kPa), inlet and outlet flow (L/min), cumulative pulse counts and 36 acoustic features (RMS, peak, peak-to-peak, standard deviation, energy, crest factor, hit count, event duration and dominant frequency for each of four channels). Each run has a JSON file with its label (0 = no leak, 1 = L1, 2 = L2, 3 = L3), valve setting, measured leak rate and conditions.

## 7. Machine Learning Methodology




*Figure 3. Processing pipeline from raw rows to a leak decision.*

### 7.1 Physical basis of the features

**Flow (mass conservation).** Because oil is practically incompressible, the leak flow equals the difference between inlet and outlet flow:

$$Q_{leak} = Q_{in} - Q_{out} = Q1 - Q2$$

This detects and quantifies a leak but carries no location information. **Pressure (orifice flow).** A hole behaves like an orifice, so pressure downstream of the leak drops and P1, P2 and P1 − P2 shift:

$$Q = C_d \cdot A \cdot \sqrt{(2\Delta P / \rho)}, \quad C_d \approx 0.6$$

**Acoustics (jet noise).** Oil forced through a hole forms a turbulent jet producing broadband high-frequency noise that attenuates with distance and at hose joints, so the disc nearest the leak hears it loudest; the loudness pattern across the four discs indicates the location.

## 7.2 Features

| Group | Features |
|---|---|
| Pressure (6) | P1, P2 means; ΔP = P1 − P2; standard deviations; P1 slope |
| Flow (6) | Q1, Q2 means; ΔQ; ΔQ/Q1; Q1 standard deviation; ΔP/Q1² |
| Acoustic per channel (10 × 4) | RMS, peak, peak-to-peak, std, energy, crest, hits, duration, dominant frequency, RMS variability |
| Acoustic cross-channel (8) | Total RMS; each channel's share of total RMS; log-ratios of neighbouring channels |

Ratio features remain stable when pump speed changes, which helps the model generalise across days and operating points.

## 7.3 Models

| Model | Principle | Role |
|---|---|---|
| Logistic Regression | Linear decision boundaries | Baseline |
| SVM (RBF kernel) | Non-linear maximum-margin boundary | Comparison |
| Random Forest | 300 trees on bootstrap samples and random feature subsets; majority vote | Main model |
| XGBoost | Sequential gradient-boosted trees with regularisation | Challenger |

Random Forest is preferred because it performs well on a few thousand tabular samples, needs no feature scaling, provides vote-based confidence and reports feature importance. Deep learning is not used: about seventy runs is too little data and the engineered features already encode the physics. A Random Forest regressor estimates the leak rate and is compared with the physics estimate ΔQ.

## 7.4 Validation and metrics

Windows from the same run are nearly identical, so the data are split **by run** using stratified grouped five-fold cross-validation. All models are evaluated on three feature sets — pressure only (A), pressure and flow (B), and full fusion (C) — and compared with a physics baseline that flags a leak when ΔQ exceeds its mean plus three standard deviations during normal runs. Metrics are accuracy, precision, leak recall (the priority, since a missed leak is worse than a false alarm), macro F1, the confusion matrix and leak-rate mean absolute error. The minimum detectable leak rate is taken as 3σ of ΔQ during no-leak runs.

### 7.5 Alarm logic

An alarm is raised only after four consecutive windows (two seconds) with leak probability of at least 60 %. Together with training runs that include pump speed changes, valve movements and knocks on the pipe, this rejects transient false alarms at the cost of a two-second delay.

## 8. Experimental Protocol

About seventy runs of 90 seconds are recorded in VG32 oil over three to four days, with run order mixed across days:

| Condition | Label | Variations | Runs |
|---|---|---|---|
| Normal operation | 0 | Pump 50/75/100 %, start/stop, pressure-switch cut-off, knocks, people walking | ≈25 |
| L1 centre valve | 1 | Valve ¼, ½, full × 4–5 repeats | 12–15 |
| L2 right hole | 2 | Pump 75/100 % × 4–5 repeats | 12–15 |
| L3 left hole | 3 | Pump 75/100 % × 4–5 repeats | 12–15 |

Before each session the pressure readings and acoustic baselines are tared with the pump off. During each leak run, the leaked oil is collected in a measuring jug for 30 seconds to give the true leak rate. Only one leak is open at a time, and every run is entered in a shared lab sheet.

## 9. Demonstration Procedure

1. Without hardware: install the Python requirements, run simulate.py, train.py and the dashboard in replay mode; the dashboard raises the correct leak alarm for each simulated run.
2. On the rig: flash the ESP32 and STM32 firmware, set the serial port and measured positions in config.py, tare with the pump off, and train on real data.
3. Live demo: start the pump at 100 %; the dashboard shows normal operation. Open L1 and within about two seconds it shows a leak at L1 with the estimated leak rate; close it and repeat with L2 and L3.
4. Discrimination: change the pump speed or knock the pipe; no alarm should be raised.

The full step-by-step guide is in docs/07_demo_guide.md of the repository.

## 10. Project Plan and Status

| Week | Phase | Gate |
|---|---|---|
| 1 | Commission rig with water; test software on simulator | No drips; software chain works |
| 1–2 | Pressure and flow electronics; flow calibration | Q1 − Q2 rises when L1 opens |
| 2 | One acoustic channel on oscilloscope | Leak visible vs. closed valve |
| 3 | Four acoustic channels; STM32 firmware; ESP32 merge | Live 43-column rows |
| 4 | Data collection in VG32 oil | ≈70 labelled runs |
| 5 | Training, demo, report | A/B/C results; rehearsed demo |

| Part | Status |
|---|---|
| Hydraulic loop | Built |
| Python pipeline | Written; tested on simulated data |
| ESP32 firmware | Written; hardware testing pending |
| STM32 acoustic firmware | Specified; to be implemented |
| Leak points L2, L3 and PZT saddles | To be built |
| Real data and results | Pending |

## 11. Scope and Limitations

- Leaks are localised to trained positions (L1–L3), not arbitrary points.
- The PZT discs are resonant 40 kHz transducers, not calibrated acoustic-emission sensors; acoustic features are relative measurements.
- The STM32 samples channels sequentially with a single converter, which supports energy-based localisation but not time-difference-of-arrival methods.
- Results from simulated data verify the software only and are not experimental findings.

## 12. References

[1] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD, 2016, pp. 785–794.

[3] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[4] M. A. Adegboye, W.-K. Fung and A. Karnik, “Recent advances in pipeline monitoring and oil leakage detection technologies,” Sensors, vol. 19, no. 11, p. 2548, 2019.

[5] R. B. Randall, Vibration-based Condition Monitoring, 2nd ed. Wiley, 2011.

[6] F. M. White, Fluid Mechanics. McGraw-Hill.

[7] Analog Devices, AD620 Low Cost, Low Power Instrumentation Amplifier, datasheet.

[8] Texas Instruments, ADS111x Ultra-Small, Low-Power, I²C-Compatible 16-Bit ADCs, datasheet.

[9] Espressif Systems, ESP32 Series Datasheet.

[10] STMicroelectronics, STM32F411xC/xE Datasheet.